

Data Analysis with Python 2024–2025

Parte 5: Pandas (Parte 3)

Soluções dos Exercícios

Soluções independentes elaboradas para fins de estudo, com base nos enunciados originais

Nota Importante

Este material é uma adaptação das soluções independentes para os exercícios baseados no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**. As soluções abaixo não são o gabarito oficial do curso.

Conteúdo

1 Soluções - Pandas (Parte 3)

Exercício 5.1 – Divisão de Data Continua

Solução proposta

```
import pandas as pd

def split_date_continues():
    df = pd.read_csv("src/Helsingin_pyorailijamaarat.csv", sep=";")
    df = df.dropna(axis=0, how="all")
    df = df.dropna(axis=1, how="all")

    # Criando o DataFrame das datas
    dates = df["Päivämäärä"].str.split(expand=True)
    dates.columns = ["Weekday", "Day", "Month", "Year", "Hour"]

    # Dicionario para normalizar o mes
    month_map = {"tammi": 1, "helmi": 2, "maalis": 3, "huhti": 4, "touko": 5, "kesä": 6, "heinä": 7, "elo": 8, "syys": 9, "loka": 10, "marras": 11, "joulu": 12}
    dates["Month"] = dates["Month"].map(month_map)
    dates["Year"] = dates["Year"].astype(int)
    dates["Day"] = dates["Day"].astype(int)
    dates["Hour"] = dates["Hour"].str.split(":", expand=True)[0].astype(int)

    df = df.drop(columns=["Päivämäärä"])
    return pd.concat([dates, df], axis=1)
```

Esta solução retoma o processamento de texto feito nas partes anteriores, e concatena o novo DataFrame de colunas temporais com as medições limpas usando `axis=1`.

Exercício 5.2 – Clima de Ciclismo

Solução proposta

```
def cycling_weather():
    cycling = split_date_continues()
    weather = pd.read_csv("src/kumpula-weather-2017.csv")

    merged = pd.merge(cycling, weather, left_on=["Year", "Month", "Day"], right_on=["Year", "m", "d"])
    merged = merged.drop(columns=["m", "d", "Time", "Time zone"])
    return merged
```

O método `pd.merge` é acionado usando as chaves de ano, mês e dia. Como no DataFrame climático o mês e o dia são denotados por 'm' e 'd', informamos ao pandas essas referências diferentes utilizando `left_on` e `right_on`.

Exercício 5.3 – Top Bandas

Solução proposta

```
def top_bands():
    top40 = pd.read_csv("src/UK-top40-1964-1-2.tsv", sep="\t")
    bands = pd.read_csv("src/bands.tsv", sep="\t")

    merged = pd.merge(top40, bands, left_on=["Artist"], right_on=["
        Band"])
    return merged
```

Exercício 5.4 – Ciclistas por Dia

Solução proposta

```
import matplotlib.pyplot as plt

def cyclists_per_day():
    df = split_date_continues()
    df = df.drop(columns=["Hour", "Weekday"])
    return df.groupby(["Year", "Month", "Day"]).sum()

def main():
    df_daily = cyclists_per_day()

    # Restringir a Agosto (Mes=8) do ano 2017
    august = df_daily.loc[(2017, 8, slice(None))]

    # Opcional: ajustar o indice para ter apenas o Dia para o eixo
    # x
    august = august.reset_index(level=[0, 1], drop=True)

    august.plot(figsize=(10,6))
    plt.xlabel("Dia")
    plt.xticks(range(1, 32))
    plt.show()

if __name__ == "__main__":
    main()
```

Através do `groupby` junto do `sum()`, obtivemos o somatório das observações diárias e o DataFrame ficou com um índice múltiplo (Ano, Mês, Dia). O `loc` permite acessar o recorte de 2017 e Mês 8 com `slice(None)` abrangendo todos os dias.

Exercício 5.5 – Melhor Gravadora

Solução proposta

```
def best_record_company():
    df = pd.read_csv("src/UK-top40-1964-1-2.tsv", sep="\t")

    # Encontrar a gravadora que soma o maior valor na coluna WoC
    best_publisher = df.groupby("Publisher")["WoC"].sum().idxmax()

    return df[df["Publisher"] == best_publisher]
```

Com `df.groupby("Publisher")["WoC"].sum()` obtemos a soma de semanas para cada gravadora. A aplicação encadeada de `idxmax()` retorna a chave (nome da gravadora) com o maior valor.

Exercício 5.8 – Séries Temporais de Bicicletas

Solução proposta

```
def bicycle_timeseries():
    df = split_date_continues()

    # Criar string unificada de data e hora
    datetime_str = df["Year"].astype(str) + "-" + df["Month"].\
        astype(str) + "-" + \
        df["Day"].astype(str) + " " + df["Hour"].astype(\
            str) + ":00:00"

    df["Date"] = pd.to_datetime(datetime_str)
    df = df.set_index("Date")
    df = df.drop(columns=["Year", "Month", "Day", "Hour", "Weekday"])

    return df
```

Aproveitamos as informações divididas no exercício 5.1 para juntá-las num formato reconhecível por `pd.to_datetime`. Em seguida, usamos `set_index("Date")` e varremos fora as colunas individuais temporais.

Exercício 5.9 – Deslocamento (Commute)

Solução proposta

```
def commute():
    df = bicycle_timeseries()

    # Filtrar Agosto de 2017
    august = df["2017-08-01":"2017-08-31"]

    # Adicionar novamente a coluna de dias da semana (onde 1=
    Segunda, 7=Domingo)
    august["Weekday"] = august.index.dayofweek + 1

    # Agrupar pelo dia da semana e agregar pela soma
    commute_df = august.groupby("Weekday").sum()

    return commute_df

def main():
    commute_df = commute()
    commute_df.plot(figsize=(8,5))
    weekdays = ["Seg", "Ter", "Qua", "Qui", "Sex", "Sab", "Dom"]
    plt.xticks(range(1, 8), weekdays)
    plt.show()

if __name__ == "__main__":
    main()
```

No `DatetimeIndex`, `dayofweek` retorna valores de 0 a 6. Somando 1, formatamos o recorte entre 1 (Segunda) e 7 (Domingo). O `groupby` fará a união estatística somando as contagens diárias de todos os dias compatíveis de Agosto.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.